

Intro to NLP (097215) - Tutorial 7

This notebook follows Andrej Karpathy's [Let's build the GPT Tokenizer](#) Youtube video.

Tokenization webapp

Good tokenization web app: <https://tiktokenizer.vercel.app>

Unicode

[A Programmer's Introduction to Unicode](#)

[UTF-8 Everywhere](#)

```
[ord(x) for x in "hello"]
```

```
[104, 101, 108, 108, 111]
```

```
chr(104)
```

```
'h'
```

```
[ord(x) for x in "הי לך"]
```

```
[1513, 1500, 1493, 1501]
```

Code cell output actions

```
[ord(x) for x in "א "]
```

```
[21644, 24179]
```

```
print(list("h".encode("utf-8")))
print(list("h".encode("utf-16")))
print(list("h".encode("utf-32")))
print(list("hello".encode("utf-8")))
print(list("hello".encode("utf-16")))
print(list("hello".encode("utf-32")))
print(list("🐶".encode("utf-8")))
```

```
[104]
[255, 254, 104, 0]
[255, 254, 0, 0, 104, 0, 0, 0]
[104, 101, 108, 108, 111]
[255, 254, 104, 0, 101, 0, 108, 0, 108, 0, 111, 0]
[255, 254, 0, 0, 104, 0, 0, 0, 101, 0, 0, 0, 108, 0, 0, 0, 108, 0, 0, 0, 111, 0, 0, 0]
[240, 159, 152, 132]
```

```
print(list("הי לך".encode("utf-8")))
print(list("א ".encode("utf-8")))
```

```
[215, 169, 215, 156, 215, 149, 215, 157]
[229, 146, 140, 229, 185, 179]
```

```
# text from https://www.reedbeta.com/blog/programmers-intro-to-unicode/
text = "🐶 hello"
tokens = text.encode("utf-8") # raw bytes
tokens = list(map(int, tokens)) # convert to a list of integers in range 0..255 for convenience
print('---')
print(list(tokens))
print('---')
print(text)
print("length:", len(text))
print('---')
print(tokens)
print("length:", len(tokens))
```

```
---
[240, 159, 152, 132, 32, 104, 101, 108, 108, 111]
---
```

```
🐶 hello
length: 7
---
```

```
[240, 159, 152, 132, 32, 104, 101, 108, 108, 111]
length: 10
```

```
# text from https://www.reedbeta.com/blog/programmers-intro-to-unicode/
text = "          ! UNICODE? U N I C O D E ! 😬 The very name strikes fear and awe into the hearts of pr
tokens = text.encode("utf-8") # raw bytes
tokens = list(map(int, tokens)) # convert to a list of integers in range 0..255 for convenience
print('----')
print(text)
print("length:", len(text))
print('----')
print(tokens)
print("length:", len(tokens))
```

```
----
          ! UNICODE? U N I C O D E ! 😬 The very name strikes fear and awe into the hearts of programmers
length: 533
----
[239, 188, 181, 239, 189, 142, 239, 189, 137, 239, 189, 131, 239, 189, 143, 239, 189, 132, 239, 189, 133, 33, 32,
length: 616
```

Byte-pair encoding

```
def get_stats(ids):
    counts = {}
    for pair in zip(ids, ids[1:]): # Pythonic way to iterate consecutive elements
        counts[pair] = counts.get(pair, 0) + 1
    return counts
```

```
stats = get_stats(tokens)
# print(stats)
print(sorted((v,k) for k,v in stats.items()). reverse=True))
```

Code cell output actions

```
[(20, (101, 32)), (15, (240, 159)), (12, (226, 128)), (12, (105, 110)), (10, (115, 32)), (10, (97, 110)), (10, (3
```

```
top_pair = max(stats, key=stats.get)
top_pair
```

```
(101, 32)
```

```
[bytes([idx]) for idx in top_pair]
```

```
[b'e', b' ']
```

```
def merge(ids, pair, idx):
    # in the list of ints (ids), replace all consecutive occurrences of pair with the new token idx
    newids = []
    i = 0
    while i < len(ids):
        # if we are not at the very last position AND the pair matches, replace it
        if i < len(ids) - 1 and ids[i] == pair[0] and ids[i+1] == pair[1]:
            newids.append(idx)
            i += 2
        else:
            newids.append(ids[i])
            i += 1
    return newids
```

```
print(merge([5, 6, 6, 7, 9, 1], (6, 7), 99))
```

```
tokens2 = merge(tokens, top_pair, 256)
print(tokens2)
print("length:", len(tokens2))
```

```
[5, 6, 99, 9, 1]
[239, 188, 181, 239, 189, 142, 239, 189, 137, 239, 189, 131, 239, 189, 143, 239, 189, 132, 239, 189, 133, 33, 32,
length: 596
```

Full tokenizer training text

```
# @title Full tokenizer training text
```

```
# making the training text longer to have more representative token statistics
```

```
# text from https://www.reedbeta.com/blog/programmers-intro-to-unicode/
```

```
text = """"A Programmer's Introduction to Unicode March 3, 2017 · Coding · 22 Comments          ! UNICODE?
U \u200cN \u200cI \u200cC \u200cO \u200cD \u200cE ! 😬 The very name strikes fear and awe into the hearts of pro
```

We all know we ought to support Unicode in our software (whatever that means—like using `wchar_t` for all the strings). Unicode can be abstruse, and diving into the thousand-page Unicode Standard plus its dozens of supplementary annexes can be more than a little intimidating. I don't blame programmers for still finding the whole thing mysterious after Unicode's inception. A few months ago, I got interested in Unicode and decided to spend some time learning detail. In this article, I'll give an introduction to it from a programmer's point of view. I'm going to focus on what's involved in working with strings and files of Unicode text. However, in this article I'm not going to talk about text layout/shaping/rendering, or localization in detail—those are separate issues, beyond my scope (and knowledge). And Inherent Complexity The Unicode Codespace Codespace Allocation Scripts Usage Frequency Encodings UTF-8 UTF-16 Canonical Equivalence Normalization Forms Grapheme Clusters And More... Diversity and Inherent Complexity As soon as you start looking at Unicode, it becomes clear that it represents a large jump in complexity over character sets like ASCII that you might be used to. It's not just that Unicode contains a much larger number of characters, although that's part of it. Unicode has a deal of internal structure, features, and special cases, making it much more than what one might expect a mere "character set". We'll see some of that later in this article. When confronting all this complexity, especially as an engineer, it's not unusual to find oneself asking, "Why do we need all this? Is this really necessary? Couldn't it be simplified?" However, Unicode is designed to represent the entire world's writing systems. The Unicode Consortium's stated goal is "enabling people around the world to use computers in any language". And as you might imagine, the diversity of written languages is immense! To date, Unicode supports over 1100 different scripts, covering some 1100 languages, and there's still a long tail of over 100 unsupported scripts, but many of those are historical, which people are still working to add. Given this enormous diversity, it's inevitable that representing all this is a complicated project. Unicode embraces that diversity, and accepts the complexity inherent in its mission to include all the world's writing systems. It doesn't make a lot of trade-offs in the name of simplification, and it makes exceptions to its own rules to further its mission. Moreover, Unicode is committed not just to supporting texts in any single language, but to supporting multiple languages coexist within one text—which introduces even more complexity. Most programming languages have been designed to be available to handle the gory low-level details of text manipulation, but as a programmer, you'll still need to know the details of Unicode features in order to know when and how to apply them. It may take some time to wrap your head around it, but it's worth it—discouraged—think about the billions of people for whom your software will be more accessible through supporting Unicode. Embrace the complexity! The Unicode Codespace Let's start with some general orientation. The basic element of Unicode is the "character", although that term isn't quite right—are called code points. Code points are identified by number, usually in hexadecimal with the prefix "U+", such as U+0041 "A" latin capital letter A or U+03B8 "θ" greek small letter theta. Each code point also has a short name, and quite a few other properties, specified in the Unicode Character Database. The space of all possible code points is called the codespace. The Unicode codespace consists of 1,114,112 code points. However, only 128,256 of the codespace—are actually assigned, to date. There's plenty of room for growth! Unicode also reserves an additional 1,048,856 code points as "private-use" areas, which have no standardized meaning and are available for individual applications to use for their own purposes. Codespace Allocation To get a feel for how the codespace is laid out, it's helpful to visualize it. I've created a map of the entire codespace, with one pixel per code point. It's arranged in tiles for visual coherence; each small square represents one code point, and each large square is a "plane" of 65,536 code points. There are 17 planes altogether. Map of the Unicode Codespace (click to zoom) White represents unassigned code points. Blue is assigned code points, green is private-use areas, and yellow is surrogates (more about those later). As you can see, the assigned code points are distributed somewhat sparsely across the codespace. In the first three planes. Plane 0 is also known as the "Basic Multilingual Plane", or BMP. The BMP contains essentially all the characters needed for modern text in any script, including Latin, Cyrillic, Greek, Han (Chinese), Japanese, Korean, Devanagari (Indian), and many more. (In the past, the codespace was just the BMP and no more—Unicode was originally designed as a straightforward 16-bit encoding, with only 65,536 code points. It was expanded to its current size in 1996. However, the vast majority of code points in modern text belong to the BMP.) Plane 1 contains historical scripts, such as Sumerian, Egyptian hieroglyphs, as well as emoji and various other symbols. Plane 2 contains a large block of less-common characters. The remaining planes are empty, except for a small number of rarely-used formatting characters in Plane 3, which are reserved entirely for private use. Scripts Let's zoom in on the first three planes, since that's where the vast majority of scripts in Unicode live. This map color-codes the 135 different scripts in Unicode. You can see that Korean (Hangeul) take up most of the range of the BMP (the left large square). By contrast, all of the European, Middle Eastern, and Asian scripts fit into the first row of the BMP in this diagram. Many areas of the codespace are adapted or copied from existing encodings. For example, the first 128 code points of Unicode are just a copy of ASCII. This has clear benefits for interoperability—easy to losslessly convert texts from smaller encodings into Unicode (and the other direction too, as long as no characters from the smaller encoding are used). Usage Frequency One more interesting way to visualize the codespace is to look at how often each code point is actually used in real-world texts. Below is a heat map of code point usage frequency in Unicode planes 0–2 (click to zoom). You can see that the vast majority of code point usage lies in the BMP, with only scattered usage of code points from planes 1–2. The biggest exception is emoji, which are located in the several bright squares in the bottom row of plane 1. Encodings We've seen that Unicode code points are identified by their index in the codespace, ranging from U+0000 to U+10FFFF. But how do code points get represented in memory or in a file? The most convenient, computer-friendliest (and programmer-friendliest) thing to do would be to represent each code point index as a 32-bit integer. This works, but it consumes 4 bytes per code point, which is sort of a lot. For Unicode will cost you a bunch of extra storage, memory, and performance in bandwidth-bound scenarios, if you're dealing with large amounts of text. Consequently, there are several more-compact encodings for Unicode. The 32-bit integer encoding is official (UTF = "Unicode Transformation Format"), but it's rarely used for storage. At most, it comes up sometimes as a temporary representation, for examining or operating on the code points in a string. Much more commonly, you'll see Unicode text encoded either UTF-8 or UTF-16. These are both variable-length encodings, made up of 8-bit or 16-bit units, respectively. UTF-8 UTF-8 code points with smaller index values take up fewer bytes, which saves a lot of memory for typical texts. The transformation from UTF-8 to UTF-16 is more programmatically involved, and likely slower. UTF-8 In UTF-8, each code point is encoded using 1 to 4 bytes, based on its index value. UTF-8 uses a system of binary prefixes, in which the high bits of each byte mark the beginning of a single byte, the beginning of a multi-byte sequence, or a continuation byte; the remaining bits, concatenated, give the index value. This table shows how it works: UTF-8 (binary) \tCode point (binary) \tRange 0xxxxxxx\txxxxxxx\tU+0000–U+007F 10yyyyyy\txxxxxyyy\tU+0080–U+07FF 1110xxxx\t0yyyyyy\t10zzzzzz\txxxxxyyy\tU+0800–U+FFFF 11110xxx\t0yyyyyy\t10zzzzzz\txxxxxyyy\tU+10000–U+10FFFF A handy property of UTF-8 is that code points below 128 (ASCII) are encoded as single bytes, and all non-ASCII code points are encoded using sequences of bytes 128–255. This has a couple of consequences. First, any strings or files out there that are already in ASCII can also be interpreted as UTF-8 without any conversion. Second, lots of widely-used string programming idioms—such as null termination, or delimiters (newline, slashes, etc.)—will just work on UTF-8 strings. ASCII bytes never occur inside the encoding of non-ASCII code points, so a byte-wise for a null terminator or a delimiter will do the right thing. Thanks to this convenience, it's relatively easy for legacy ASCII programs and APIs to handle UTF-8 strings. UTF-8 is very widely used in the Unix/Linux and Web worlds. Many programmers argue UTF-8 should be the default encoding everywhere. However, UTF-8 isn't a drop-in replacement for all respects. For instance, code that iterates over the "characters" in a string will need to decode UTF-8 and interpret the code points (or maybe grapheme clusters—more about those later), not bytes. When you measure the "length" of a string, you need to think about whether you want the length in bytes, the length in code points, the width of the text when rendered, or the number of grapheme clusters. UTF-16 The other encoding that you're likely to encounter is UTF-16. It uses 16-bit words, with each code point stored in one or two words. Like UTF-8, we can express the UTF-16 encoding rules in the form of binary prefixes: UTF-16 (binary) \t

\tRange xxxxxxxxxxxxxxxxxx\txxxxxxxxxxxxxxxxxxx\tU+0000-U+FFFF 110110xxxxxxxxxxx 110111yyyyyyyyyyy\txxxxxxxxxxxxxxxxxxxxxxx
 +10FFFF A more common way that people talk about UTF-16 encoding, though, is in terms of code points called “surrogate code points” in the range U+D800–U+DFFF—or in other words, the code points that match the binary prefixes 110110 and 110111 in the table above—are reserved specifically for UTF-16 encoding, and don’t represent any valid characters on their own. They only occur to occur in the 2-word encoding pattern above, which is called a “surrogate pair”. Surrogate code points are illegal in any other context! They’re not allowed in UTF-8 or UTF-32 at all. Historically, UTF-16 is a descendant of the original, pre-Unicode, in which there were only 65,536 code points. The original intention was that there would be no different characters in Unicode was supposed to be a straightforward 16-bit character set. Later, the codespace was expanded to make room for less-common (but still important) Han characters, which the Unicode designers didn’t originally plan for. Surrogates were introduced, as—to put it bluntly—a kludge, allowing 16-bit encodings to access the new code points. Today, JavaScript’s standard string representation: if you ask for the length of a string, or iterate over it, etc., the result will be the number of words, with any code points outside the BMP expressed as surrogate pairs. UTF-16 is also used by the Microsoft Windows API. Win32 supports either 8-bit or 16-bit strings, the 8-bit version unaccountably still doesn’t support UTF-8—only UTF-16 encodings, like ANSI. This leaves UTF-16 as the only way to get proper Unicode support in Windows. (Update: in Windows 10 they finally added UTF-8 support to the 8-bit APIs! 😊) By the way, UTF-16’s words can be stored either little-endian or big-endian. Unicode has no opinion on that issue, though it does encourage the convention of putting U+FEFF zero width no-break space at the beginning of a UTF-16 file as a byte-order mark, to disambiguate the endianness. (If the file doesn’t match the system’s endianness, it will be decoded as U+FFFE, which isn’t a valid code point.) Combining Marks In the story so far, we’ve been focusing on individual code points. But in Unicode, a “character” can be more complicated than just an individual code point! Unicode includes a system for composing characters, by combining multiple code points together. This is used in various ways to gain flexibility and to avoid a huge combinatorial explosion in the number of code points. In European languages, for example, this shows up in the use of diacritics to letters. Unicode supports a wide range of diacritics, including acute and grave accents, umlauts, and more. All these diacritics can be applied to any letter of any alphabet—and in fact, multiple diacritics can be applied to a single letter. If Unicode tried to assign a distinct code point to every possible combination of letter and diacritics, they would rapidly get out of hand. Instead, the dynamic composition system enables you to construct the character you want, by taking a base code point (the letter) and appending additional code points, called “combining marks”, to specify the diacritics. When a renderer sees a sequence like this in a string, it automatically stacks the diacritics over or under the base letter. For example, the accented character “Á” can be expressed as a string of two code points: U+0041 latin capital letter A plus U+0301 “◌́” combining acute accent. This string automatically gets rendered as a single character. Unicode does also include many “precomposed” code points, each representing a letter with some combination of diacritics already applied, such as U+00C1 “Á” latin capital letter A with acute or U+00C7 “Ç” latin capital letter C with cedilla. We suspect these are mostly inherited from older encodings that were assimilated into Unicode, and kept around for compatibility. In practice, there are precomposed code points for most of the common letter-with-diacritic combinations in European languages, so they don’t use dynamic composition that much in typical text. Still, the system of combining marks does allow an arbitrary number of diacritics to be stacked on any base character. The reductio-ad-absurdum of this is Zalgo text, which was created by randomly stacking numerous diacritics on each letter, so that they overflow the line spacing and create an appearance of corruption. A few other places where dynamic character composition is used are in Arabic and Hebrew. In these languages, words are normally spelled with vowels left out. They then have diacritic notation to indicate the vowels (used in dictionaries, language-teaching materials, children’s books, and such). These diacritics are expressed with combining marks. A Hebrew example, with niqqud: נִשְׁכַּח הַיָּד הַיְמָנִית, קֶטֶב לְשִׁכְחֵי יְשׁוּדִים Normal writing (no niqqud): נשכח היד הימנית, קטב לשכחי ישודים Devanagari, the script used to write Hindi, Sanskrit, and many other South Asian languages, expresses certain vowels as combining marks attached to letters. For example, “ह” + “\u200b” = “ह्” (“h” + “i” = “hi”). Korean characters stand for syllables, but they also have letters called jamo that stand for the vowels and consonants in the syllable. While there are code points for precomposed syllables, it’s also possible to dynamically compose them by concatenating their jamo. For example, “ㄱ” + “ㅏ” + “ㄴ” = “한” (“han”). Canonical Equivalence In Unicode, precomposed characters exist alongside the dynamic composition system. The consequence of this is that there are multiple ways to express “the same” string—different sequences of code points that are perceived as the same user-perceived character. For example, as we saw earlier, we can express the character “Á” either as the single code point U+00C1, or as the string of two code points U+0041 U+0301. Another source of ambiguity is the ordering of multiple diacritics on a single character. Diacritic order matters visually when two diacritics apply to the same side of the base character. For example, “ā” (dot, then macron) is different from “ȃ” (macron, then dot). However, when diacritics apply to different sides, e.g. one above and one below, then the order doesn’t affect rendering. Moreover, a character with multiple diacritics can be expressed in five different ways: Fully precomposed: U+1E77 “ē” Partially precomposed: U+1E79 “e” + U+0302 “◌̂” Partially precomposed: U+0045 “e” + U+0302 “◌̂” Fully decomposed: U+0065 “e” + U+0302 “◌̂” Fully decomposed: U+0065 “e” + U+0302 “◌̂” + U+0303 “◌̃” Unicode refers to set of strings like this as “canonically equivalent”. Canonically equivalent strings are supposed to be identical for purposes of searching, sorting, rendering, text selection, and so on. This has implications for how applications operate on text. For example, if an app has a “find in file” operation and the user searches for “ē”, it should find all occurrences of any of the five versions of “ē” above! Normalization Forms To address the problem of “how to handle canonically equivalent strings”, Unicode defines several normalization forms: ways of converting strings into a canonical form so that they can be compared code-point-by-code-point (or byte-by-byte). The “NFD” normalization form fully decomposes every character into its component base and combining marks, taking apart any precomposed code points in the string. It also sorts the combining marks according to their rendered position, so e.g. diacritics that go below the character come before the ones that go above. (It doesn’t reorder diacritics in the same rendered position, since their order matters visually, as previously mentioned.) The “NFC” form, conversely, puts things back together into precomposed code points as much as possible. A combination of diacritics is called for, there may not be any precomposed code point for it, in which case NFC still uses the decomposed form. It can and leaves any remaining combining marks in place (again ordered by rendered position, as in NFD). There are also two forms called NFKD and NFKC. The “K” here refers to compatibility decompositions, which cover characters that are “similar” but not visually identical. However, I’m not going to cover that here. Grapheme Clusters As we’ve seen, Unicode includes a way to handle cases where a thing that a user thinks of as a single “character” might actually be made up of multiple code points. Unicode formalizes this using the notion of a grapheme cluster: a string of one or more code points that constitute a “user-perceived character”. UAX #29 defines the rules for what, precisely, qualifies as a grapheme cluster. It’s defined as a base code point followed by any number of combining marks, but the actual definition is a bit more complicated; it also covers things like Korean jamo, and emoji ZWJ sequences. The main thing grapheme clusters are used for is text editing: they’re the most sensible unit for cursor placement and text selection boundaries. Using grapheme clusters for these purposes can’t accidentally chop off some diacritics when you copy-and-paste text, that left/right arrow keys always move by whole grapheme clusters, and so on. Another place where grapheme clusters are useful is in enforcing a string length limit on a database field. While the true, underlying limit might be something like the byte length of the string in UTF-8, it’s often easier to enforce that by just truncating bytes. At a minimum, you’d want to “round down” to the nearest code point boundary.

round down to the nearest grapheme cluster boundary. Otherwise, you might be corrupting the last character by cutting a diacritic, or interrupting a jamo sequence or ZWJ sequence. And More... There's much more that could be said about the programmer's perspective! I haven't gotten into such fun topics as case mapping, collation, compatibility decomposition, confusables, Unicode-aware regexes, or bidirectional text. Nor have I said anything yet about implementation issues like efficiently storing and look-up data about the sparsely-assigned code points, or how to optimize UTF-8 decoding, stream normalization. Perhaps I'll return to some of those things in future posts. Unicode is a fascinating and complex system with a many-to-one mapping between bytes and code points, and on top of that a many-to-one (or, under some circumstances, one-to-many) mapping between code points and "characters". It has oddball special cases in every corner. But no one ever claims that writing all written languages was going to be easy, and it's clear that we're never going back to the bad old days of a proliferation of incompatible encodings. Further reading: The Unicode Standard UTF-8 Everywhere Manifesto Dark corners of Unicode (International Components for Unicode)-C/C++/Java libraries implementing many Unicode algorithms and related things Howto Google Noto Fonts-set of fonts intended to cover all assigned code points""

```
tokens = text.encode("utf-8") # raw bytes
tokens = list(map(int, tokens)) # convert to a list of integers in range 0..255 for convenience
```

```
len(tokens)
```

```
24597
```

✧ Training the tokenizer using Byte-Pair Encoding

```
def get_stats(ids):
    counts = {}
    for pair in zip(ids, ids[1:]):
        counts[pair] = counts.get(pair, 0) + 1
    return counts

def merge(ids, pair, idx):
    newids = []
    i = 0
    while i < len(ids):
        if i < len(ids) - 1 and ids[i] == pair[0] and ids[i+1] == pair[1]:
            newids.append(idx)
            i += 2
        else:
            newids.append(ids[i])
            i += 1
    return newids

# ---
vocab_size = 276 # the desired final vocabulary size
num_merges = vocab_size - 256
ids = list(tokens) # copy so we don't destroy the original list

merges = {} # (int, int) -> int
for i in range(num_merges):
    stats = get_stats(ids)
    pair = max(stats, key=stats.get)
    idx = 256 + i
    print(f"merging {pair} into a new token {idx}")
    ids = merge(ids, pair, idx)
    merges[pair] = idx
```

```
merging (101, 32) into a new token 256
merging (105, 110) into a new token 257
merging (115, 32) into a new token 258
merging (116, 104) into a new token 259
merging (101, 114) into a new token 260
merging (99, 111) into a new token 261
merging (116, 32) into a new token 262
merging (226, 128) into a new token 263
merging (44, 32) into a new token 264
merging (97, 110) into a new token 265
merging (111, 114) into a new token 266
merging (100, 32) into a new token 267
merging (97, 114) into a new token 268
merging (101, 110) into a new token 269
merging (257, 103) into a new token 270
merging (261, 100) into a new token 271
merging (121, 32) into a new token 272
merging (46, 32) into a new token 273
merging (97, 108) into a new token 274
merging (259, 256) into a new token 275
```

```
print("tokens length:", len(tokens))
print("ids length:", len(ids))
print(f"compression ratio: {len(tokens) / len(ids):.2f}X")
```

```
tokens length: 24597
ids length: 19438
```

compression ratio: 1.27X

▼ Vocab

```
print(merges)
```

```
{(101, 32): 256, (105, 110): 257, (115, 32): 258, (116, 104): 259, (101, 114): 260, (99, 111): 261, (116, 32): 262,
```

```
# Given a sequence of integers in the range [0, vocab_size], what is the text?
```

```
vocab = {idx: bytes([idx]) for idx in range(256)}
for (p0, p1), idx in merges.items():
    vocab[idx] = vocab[p0] + vocab[p1]
```

```
print(vocab)
```

```
{0: b'\x00', 1: b'\x01', 2: b'\x02', 3: b'\x03', 4: b'\x04', 5: b'\x05', 6: b'\x06', 7: b'\x07', 8: b'\x08', 9: b'\x09',
```

```
print(b'a'+b'c')
```

```
b'ac'
```

▼ Decoder

```
def decode(ids):
    # given ids (list of integers), return Python string
    tokens = b"".join(vocab[idx] for idx in ids)
    text = tokens.decode("utf-8", errors="replace")
    return text
```

```
print(decode([269, 265]))
```

```
enan
```

▼ Encoder

```
merges
```

```
{(101, 32): 256,
 (105, 110): 257,
 (115, 32): 258,
 (116, 104): 259,
 (101, 114): 260,
 (99, 111): 261,
 (116, 32): 262,
 (226, 128): 263,
 (44, 32): 264,
 (97, 110): 265,
 (111, 114): 266,
 (100, 32): 267,
 (97, 114): 268,
 (101, 110): 269,
 (257, 103): 270,
 (261, 100): 271,
 (121, 32): 272,
 (46, 32): 273,
 (97, 108): 274,
 (259, 256): 275}
```

```
def encode(text):
    # Encode the input text as a list of UTF-8 bytes (integers).
    # This initial list represents the initial tokens, each corresponding to a single character.
    tokens = list(text.encode("utf-8"))

    # Continue attempting to merge tokens as long as there are at least two tokens left.
    # This allows for the possibility of further merging.
    while len(tokens) >= 2:
        # Calculate the frequency of adjacent pairs of tokens in the current list of tokens.
        stats = get_stats(tokens)
```

```
# Find the pair with the lowest index in the 'merges' dictionary, indicating the next merge.
# If a pair doesn't exist in 'merges', assign it an infinite index to prioritize pairs that can be merge
pair = min(stats, key=lambda p: merges.get(p, float("inf")))

# If the chosen pair does not exist in the merges dictionary, no further merging is possible.
# Exit the loop to prevent infinite iteration.
if pair not in merges:
    break

# Retrieve the index that represents the merged pair from the merges dictionary.
idx = merges[pair]

# Merge the identified pair throughout the tokens, replacing occurrences of the pair with the new token.
tokens = merge(tokens, pair, idx)

# Return the final list of tokens after all possible merges have been applied.
return tokens
```

```
print(decode(encode("hello world")))
```

```
hello world
```

```
text2 = decode(encode(text))
print(text2 == text)
```

```
True
```

Code cell output actions